

Tracing Circuits in Qwen3: A Faithful Reimplementation and Reproduction of Attribution Graphs*

Zixuan Wang
zw499@cam.ac.uk

July 10, 2026

Abstract

Attribution graphs decompose a language model’s forward pass on a single prompt into causal, weighted interactions between sparse dictionary features, and have been used to map the “biology” of a frontier model. We reimplement the full methodology from scratch for the Qwen3 model family — local replacement models, batched edge attribution, influence-based pruning, and constrained-patching interventions — using the open-source circuit-tracer library only to load pretrained per-layer transcoders. The reimplementation is verified numerically equivalent to circuit-tracer: identical node sets, matching edge weights, identical pruned graphs, and matching constrained-steering effects on shared inputs. We then reproduce two case studies from Lindsey et al. [3] on Qwen3-4B — two-digit addition and multilingual antonyms — following the paper’s intervention protocols, and add a same-recipe cross-scale comparison (4B vs. 8B). The core circuit structures reproduce; several interventions reproduce only at reduced steering strength; and one causal claim (the language swap) localises differently in Qwen3, which we characterise mechanistically.

1 Introduction

Mechanistic interpretability aims to reverse-engineer the computations of neural networks into human-understandable algorithms [4, 5]. For large language models a central obstacle is that the natural unit of computation is not the neuron: models appear to represent many more concepts than they have neurons, storing them in *superposition*, so individual neurons are polysemantic and resist interpretation [6]. Explaining a behaviour mechanistically therefore requires both a better feature basis and a way to trace, on concrete prompts, how those features causally interact.

Dictionary learning addresses the first requirement. Sparse autoencoders (SAEs) decompose activations into an overcomplete basis of sparsely-firing, substantially more monosemantic features [7, 8], scale to frontier models [9], and benefit from discontinuous activation functions such as JumpReLU [10]. An SAE, however, only re-describes the representation at one point in the network: it says what is encoded, not how it is computed. *Transcoders* [11] instead train the same sparse dictionary architecture to imitate an MLP’s input–output map, so that each feature has both an input action (its encoder row) and an output action (its decoder column) and the MLP’s computation factors through an interpretable sparse bottleneck. Cross-layer variants let one feature write to all subsequent layers [1].

The second requirement — tracing interactions — has traditionally been met with causal patching. Manual activation patching over attention heads and MLPs isolated, e.g., the indirect-object-identification circuit in GPT-2 [12]; ACDC automated the search [13]; attribution patching replaced interventions with cheap gradient approximations [14]; and sparse feature circuits lifted patching from model components to SAE features [15]. *Attribution graphs* [1] sharpen this

*Code: `llm-circuits` repository. Reference method: Ameisen et al. [1]; reference implementation: Hanna et al. [2].

line of work into a per-prompt causal graph over transcoder features. Replacing MLPs with transcoders, freezing attention patterns and normalisation denominators at their on-prompt values, and inserting error nodes yields a *local replacement model* that reproduces the underlying model’s forward pass exactly while being linear in the feature activations; *direct* effects between features are then exact linear coefficients obtained by backpropagation rather than approximations, and the resulting dense graph is pruned to an interpretable subgraph whose hypotheses are validated by steering interventions on the real model. The companion paper applies the method across dozens of behaviours of Claude 3.5 Haiku [3], and the circuit-tracer library [2] open-sources the pipeline for open-weight models. Together these form the reference for the present work.

This report makes two contributions. **First**, we reimplement the entire pipeline from scratch for HuggingFace Qwen3 models [16]: capture of frozen constants, the local replacement forward pass, batched edge attribution via `autograd`, influence-based graph pruning, an interactive graph explorer, and constrained-patching feature interventions. `circuit-tracer` is used *only* to load pretrained per-layer transcoders [17]; no attribution, pruning, or intervention machinery is imported. A dedicated verification suite establishes numerical equivalence with `circuit-tracer` on shared inputs — identical node sets, edge weights matching to floating-point noise, identical influence rankings and pruned graphs, and matching constrained-steering deltas (Section 3). **Second**, we reproduce two case studies of Lindsey et al. [3] on Qwen3-4B: two-digit *addition* (parallel lookup-table and sum pathways, the operand-plot feature taxonomy, causal input suppressions, and cross-context reuse of arithmetic features) and *multilingual antonyms* (a shared cross-lingual “say-large” core with language-specific output stages, operation/operand/language swap interventions, and cross-lingual feature overlap as a function of scale, compared against Qwen3-8B with same-recipe transcoders). Where Qwen3’s tokenisation or capacity forces protocol adaptations — digit-by-digit arithmetic, chat templates, steering strengths at which a 4B model degrades — we document the adaptation explicitly and separate model differences from method failures.

Section 2 presents the method; Section 3 reports the verification and the two reproductions; Section 4 discusses what transfers across models and what does not.

2 Method

Throughout, the underlying model is a pre-norm decoder-only transformer with L layers. We write $\mathbf{x}_c^\ell \in \mathbb{R}^d$ for the residual stream at token position c before layer ℓ ’s MLP, $\tilde{\mathbf{x}}_c^\ell$ for the normalised MLP input,¹ and $\mathbf{y}_c^\ell = \text{MLP}^\ell(\tilde{\mathbf{x}}_c^\ell)$ for the MLP output added back into the stream. Logits are $\text{logit}_t = \mathbf{u}_t^\top \text{norm}(\mathbf{x}^{L+1})$ with unembedding rows $\mathbf{u}_t$.

2.1 Transcoders: sparse dictionaries for MLP computation

An SAE learns an overcomplete sparse code $\mathbf{f}(\mathbf{x}) = \sigma(W_{\text{enc}}\mathbf{x} + \mathbf{b}_{\text{enc}})$ with linear decoder $\hat{\mathbf{x}} = W_{\text{dec}}\mathbf{f} + \mathbf{b}_{\text{dec}}$, trained to reconstruct its own input under a sparsity penalty. A *transcoder* keeps this architecture but changes the target: it is trained to predict the MLP’s *output* from the MLP’s *input*, so that the learned features mediate the layer’s computation rather than merely re-encode its input. In the per-layer form (PLT) used in this work, layer ℓ has feature activations and reconstruction

$$\mathbf{a}_c^\ell = \text{JumpReLU}(W_{\text{enc}}^\ell \tilde{\mathbf{x}}_c^\ell + \mathbf{b}_{\text{enc}}^\ell), \quad \hat{\mathbf{y}}_c^\ell = W_{\text{dec}}^{\ell \top} \mathbf{a}_c^\ell + \mathbf{b}_{\text{dec}}^\ell, \quad \text{JumpReLU}(z)_i = z_i \mathbb{I}[z_i > \theta_i], \quad (1)$$

i.e. a linear map on either side of a hard sparse gate. The reference method’s default is the *cross-layer* transcoder (CLT), in which a feature encoded at layer ℓ' writes to the MLP outputs of *all* later layers through separate decoders, $\hat{\mathbf{y}}_c^\ell = \sum_{\ell' \leq \ell} W_{\text{dec}}^{\ell' \rightarrow \ell \top} \mathbf{a}_c^{\ell'} + \mathbf{b}_{\text{dec}}^\ell$; the PLT is the special

¹In Qwen3, RMSNorm: $\tilde{\mathbf{x}} = \mathbf{x} \odot \sigma(\mathbf{x}) \odot \gamma$ with $\sigma(\mathbf{x}) = \sqrt{\|\mathbf{x}\|^2/d + \varepsilon}$.

case $W_{\text{dec}}^{\ell' \rightarrow \ell} = 0$ for $\ell' \neq \ell$. Training minimises reconstruction error with a tanh sparsity penalty,

$$\mathcal{L} = \sum_{\ell=1}^L \|\hat{\mathbf{y}}^\ell - \mathbf{y}^\ell\|_2^2 + \lambda \sum_{\ell,i} \tanh(c \|W_{\text{dec},i}^\ell\| a_i^\ell), \quad (2)$$

with the JumpReLU thresholds θ trained through a straight-through estimator. The pretrained Qwen3 transcoders used here are per-layer JumpReLU dictionaries with 163,840 features per layer and no skip connection [17].

2.2 Attribution-graph generation

Local replacement model. Substituting every MLP by its transcoder yields the *replacement model*, which only approximates the underlying model. The attribution graph is instead built on the *local replacement model* for a specific prompt p : (i) MLPs are replaced by transcoders; (ii) attention patterns and normalisation denominators are *frozen* at the values they take in the underlying model’s forward pass on p , making attention and normalisation linear maps of the residual stream; and (iii) an *error node* is added at every (position, layer),

$$\mathbf{e}_c^\ell = \mathbf{y}_c^\ell - \hat{\mathbf{y}}_c^\ell, \quad (3)$$

injected as a constant so that every activation and logit of the local replacement model *exactly* equals the underlying model’s. By construction the only remaining nonlinearities are the JumpReLU gates on feature pre-activations: between any feature’s (constant-scaled) output and any downstream pre-activation, the map is affine.

Nodes. The graph has four node types: one *embedding* node per prompt token; one *feature* node per active feature instance $s = (c_s, \ell_s, i_s)$ with activation $a_s > 0$; one *error* node per (position, layer); and *logit* nodes for the smallest set of candidate output tokens whose cumulative probability reaches 0.95, capped at 10, at the final position. Embedding and error nodes have no incoming edges; logit nodes have no outgoing edges.

Edges. Edge weights are the *direct* linear effects of source on target within the local replacement model. Let $\bar{J}_{(c_s, \ell) \rightarrow (c_t, \ell_t)}$ denote the Jacobian of the residual stream at (c_t, ℓ_t) with respect to the stream at (c_s, ℓ) computed with stop-gradients on all MLP/transcoder outputs, attention patterns, and normalisation denominators — i.e. the sum over all paths through the residual stream and the frozen attention OV circuits, but through no other feature’s nonlinearity. For a feature source s and a feature target t with encoder row $W_{\text{enc}, i_t}^{\ell_t}$, the edge weight is

$$A_{s \rightarrow t} = a_s \sum_{\ell_s \leq \ell < \ell_t} (W_{\text{dec}, i_s}^{\ell_s \rightarrow \ell})^\top \bar{J}_{(c_s, \ell) \rightarrow (c_t, \ell_t)} W_{\text{enc}, i_t}^{\ell_t}, \quad (4)$$

which for the PLT collapses to the single $\ell = \ell_s$ term. Embedding and error sources use their output vectors ($\mathbf{E}_{\text{tok}}$ and $\mathbf{e}_c^\ell$) in place of $a_s W_{\text{dec}, i_s}$. For a logit target the input vector is the *vocabulary-demeaned* unembedding direction $\mathbf{u}_t - \bar{\mathbf{u}}$, applied at the (frozen-)normalised final residual — the demeaned logit is the component that affects the softmax, which is invariant to uniform shifts. Because the local replacement model is affine in the source outputs, incoming edges are *complete*: the pre-activation of any feature node satisfies $h_t = \sum_s A_{s \rightarrow t} + \text{const}$, where the constant collects bias-path terms ($\mathbf{b}_{\text{enc}}, \mathbf{b}_{\text{dec}}$) that are not represented as nodes. Attention QK circuits are deliberately outside the graph: nodes influence one another through frozen attention outputs, but effects *on* the patterns themselves are not modelled.

Computation. Edges are computed target-by-target with one backward pass per batch of targets: the target’s input vector is injected as the gradient seed at its (layer, position), the backward pass runs through the frozen-linear graph, and every source’s edge weight is the inner product of the resulting gradient at the source’s write location with the source’s output vector, as in Eq. (4). The cost is linear in the number of attributed targets and independent of the number of sources; on dense prompts, targets can be selected adaptively in decreasing order of an incrementally re-estimated influence on the logits, so that the most explanatory feature nodes are attributed first.

2.3 Graph pruning

Raw graphs contain 10^4 – 10^6 edges, most irrelevant to the output. Pruning keeps the subgraph that carries the bulk of the influence on the logits. Let A be indexed $[t, s]$ (rows = targets) and define the normalised unsigned adjacency

$$\hat{A}_{ts} = \frac{|A_{ts}|}{\sum_{s'} |A_{ts'}|}, \quad B = \sum_{k \geq 1} \hat{A}^k = (I - \hat{A})^{-1} - I, \quad (5)$$

where the Neumann series converges because the graph is a DAG ($\hat{A}$ is nilpotent); B_{tv} sums the strengths of all directed paths $v \rightarrow t$, a path’s strength being the product of its normalised edge weights. Each node v receives the probability-weighted logit influence

$$I_v = \sum_{t \in \text{logits}} p_t B_{tv}, \quad (6)$$

with p_t the model’s output probability for logit node t . **Node pruning** keeps the minimal set of nodes, in decreasing I_v , whose cumulative share of $\sum_v I_v$ reaches $\tau_{\text{node}} = 0.8$; embedding and logit nodes are always kept.² **Edge pruning** recomputes $\hat{A}'$ and I' on the pruned graph and scores each surviving edge by the influence it feeds into its target,

$$s_{ts} = \hat{A}'_{ts} (I'_t + p_t), \quad (7)$$

($p_t = 0$ for non-logit targets, and $I'_t = 0$ for logit nodes, so the parenthesis is the target’s score in either case), again keeping the minimal set with cumulative share $\tau_{\text{edge}} = 0.98$. Finally, internal nodes left dangling — features without both an incoming and an outgoing kept edge, error nodes without an outgoing one — are removed iteratively. Typical graphs shrink by roughly an order of magnitude in nodes and two in edges while retaining $\approx 80\%$ of the logit influence.

2.4 Interventions

Attribution graphs are hypotheses; they are validated by steering features in the *underlying* model and checking that downstream features and logits move as the graph predicts. Steering feature s multiplicatively by M — $a_s \mapsto M a_s$, so $M=0$ ablates and $M=-1$ flips the sign³ — adds the corresponding decoder delta to every MLP output the feature writes to:

$$\Delta \mathbf{y}_{c_s}^\ell = (M - 1) a_s W_{\text{dec}, i_s}^{\ell_s \rightarrow \ell}, \quad \ell_s \leq \ell \leq \ell_e. \quad (8)$$

Under **constrained patching** over a layer range $[\ell_s, \ell_e]$, all MLP outputs within the range are pinned to their clean recorded values before the deltas are added, so the intervention itself is the *only* change to any MLP output in the range (no second-order effects within it, matching the

²The paper’s prose also protects error nodes; the reference implementation (and ours, verified equivalent to it) lets error nodes be pruned.

³Our code parameterises the same operation by the additive multiple $m = M - 1$ of the clean activation ($m=-1$ ablate, $m=-2$ flip); circuit-tracer’s API takes the absolute target value $M a_s$. All strengths in this report are quoted in the paper’s multiplicative M convention.

graph’s direct-effect semantics); computation resumes normally after ℓ_e . Attention patterns are frozen to their clean values throughout, consistent with the graph’s frozen-attention assumption; normalisation denominators are frozen only when the range spans the whole model, in which case the intervention measures a pure direct (linear) effect on the logits. The alternative, **iterative patching**, lets downstream features re-encode their perturbed inputs and take new values, propagating knock-on effects through the real MLPs at the price of compounding reconstruction errors. Effects are read out both as changes in the output distribution and as downstream feature activations re-encoded from the perturbed forward pass, reported as percentages of their clean baselines. Because feature dictionaries are imperfect (reconstruction error, incomplete feature recruitment), steering at $|M|$ beyond the observed activation is often required before behavioural effects appear; the paper typically suppresses source features to $M = -1$ or beyond and drives donor features at several times their donor-prompt activation.

Reimplementation. Our implementation realises the above for HuggingFace Qwen3 checkpoints with per-layer transcoders: a capture pass records attention patterns, RMSNorm denominators and MLP outputs; hooks then replace MLPs with transcoder reconstructions plus constant error nodes, substitute frozen-pattern attention (V/O only) and frozen-denominator norms; batched **autograd** computes Eq. (4) for hundreds of targets per backward pass. Pruning follows Section 2.3 exactly, and interventions follow circuit-tracer’s constrained semantics. Numerical equivalence with circuit-tracer is established in Section 3.

3 Experiments and Results

[Placeholder — to be written; budgeted at $\approx 1,150$ words. Verification of the reimplementation against circuit-tracer, then the addition and multilingual reproductions from notebooks/, with per-experiment verdicts against Lindsey et al. [3].]

4 Discussion

[Placeholder — to be written; budgeted at ≈ 350 words.]

References

- [1] Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, et al. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- [2] Michael Hanna, Mateusz Piotrowski, Jack Lindsey, and Emmanuel Ameisen. circuit-tracer: An open-source library for attribution graphs, 2025. <https://github.com/decoderresearch/circuit-tracer>, v0.3.1.
- [3] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, et al. On the biology of a large language model. *Transformer Circuits Thread*, 2025. <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- [4] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. <https://distill.pub/2020/circuits/zoom-in>.

- [5] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework>.
- [6] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/toy_model.
- [7] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas L. Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features>.
- [8] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [9] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, et al. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. <https://transformer-circuits.pub/2024/scaling-monosemanticity>.
- [10] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.
- [11] Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable LLM feature circuits. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [12] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations (ICLR)*, 2023.
- [13] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [14] Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. In *Proceedings of the 7th BlackboxNLP Workshop*, 2024.
- [15] Samuel Marks, Can Rager, Eric J. Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- [16] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [17] Michael Hanna. Per-layer transcoders for the qwen3 model family, 2025. HuggingFace model repositories [mwhanna/qwen3-{0.6b,4b,8b}-transcoders](https://huggingface.co/mwhanna/qwen3-{0.6b,4b,8b}-transcoders).